

ZÁRÓDOLGOZAT

RealEstate

Ingyenbelföldi webalkalmazás

Készítette: Drótos Márk, Orosz Csaba

Helyszín: Miskolc

Év: 2026

Tartalom

A dolgozat a RealEstate nevű ingatlanhirdetési webalkalmazás fejlesztését és működését mutatja be. A projekt React + TypeScript + Vite alapú kliensből, ASP.NET Core Web API + Dapper alapú szerverből és MySQL adatbázisból épül fel. A rendszer támogatja a nyilvános hirdetésböngészést, a regisztrációt, a JWT alapú bejelentkezést, a hirdetésfeladást több képpel, a kedvencek mentését, a profilkezelést, az érdeklődések fogadását és az adminisztrációs műveleteket.

A dokumentum egységes szerkezetben tárgyalja a projektet. A szöveg a tényleges kódbázis főbb részeit követi, külön bemutatja a technológiai környezetet, a projektstruktúrát, az adatbázis-tervet, a backend és a frontend megvalósítását, a felhasználói és adminisztrátori folyamatokat, valamint a továbbfejlesztési lehetőségeket. A mellékelt képernyőképek és diagramok a rendszer felépítését és használatát szemléltetik.

Tartalomjegyzék

1. Bevezetés	3
2. A projekt célja és követelményei	3
3. Követelményanalízis	4
4. Felhasznált technológiák	5
5. Projektstruktúra és rendszerfelépítés	8
6. Adatbázisterv és adatszerkezet	10
7. Backend megvalósítás	12
8. Frontend megvalósítás	14
9. Felhasználói dokumentáció	15
10. Adminisztrátori funkciók	17
11. Működési forgatókönyvek	18
12. Biztonsági megoldások és hibakezelés	19
13. Fejlesztői környezet, indítás és tesztelés	20
14. Továbbfejlesztési lehetőségek	21
15. Összegzés	21
16. UML diagramok	22
Forrásjegyzék	25

1. Bevezetés

A dolgozatban a saját fejlesztésű RealEstate webalkalmazást mutatjuk be. A projekt kiindulópontja az volt, hogy egy olyan ingatlanhirdetési rendszert készítsünk, amely egyszerre tud nyilvános böngészést, regisztrált felhasználói funkciókat és egy alap adminisztrációs felületet biztosítani. A témaválasztás azért is volt kézenfekvő, mert az ingatlanos oldalak a gyakorlatban is sokféle problémát egyesítenek: felhasználókezelést, jogosultságkezelést, űrlapokat, összetett szűrést, képfeltöltést, adatbázis-kapcsolatot és jól áttekinthető megjelenítést.

A megvalósítás során arra törekedtünk, hogy a rendszer ne csak látványos felületet adjon, hanem ténylegesen végigvezethető adatút álljon mögötte. A frontend React + TypeScript + Vite technológiákra épül, a backend ASP.NET Core Web API alapon készült, az adatok pedig MySQL adatbázisban tárolódnak. A frontend és a backend REST alapú JSON kommunikációval kapcsolódik egymáshoz, a hitelesítés JWT tokennel történik, a jelszavak pedig BCrypt hash formában kerülnek tárolásra.

A RealEstate rendszerben a látogatók megtekinthetik az aktív hirdetéseket, szűrhetnek városra, kerületre, árra, alapterületre és további tulajdonságokra, valamint megnyithatják az egyes ingatlanok részletes adatlapját. A regisztrált felhasználók ezen felül saját hirdetést is feladhatnak, módosíthatják a profiljukat, kedvencek közé menthetnek ingatlanokat, illetve üzenetet küldhetnek egy hirdetés kapcsán. Az admin oldal külön kezelőfelületet biztosít a felhasználók, hirdetések és admin üzenetek számára.

A dolgozat célja kettős. Egyrészt szeretnénk összefoglalni a projekt felépítését és működését, másrészt be szeretnénk mutatni azokat a fejlesztői döntéseket, amelyek a rendszer jelenlegi formáját meghatározták. A dokumentáció nem egy általános elméleti mintarendszert ír le, hanem a mellékelt RealEstate projekt tényleges kódbázisa alapján készült, ezért külön hangsúlyt kap a mappaszerkezet, az alkalmazott technológiai stack, az adatbázis séma, a főbb API végpontok és a felhasználói munkafolyamatok bemutatása.

2. A projekt célja és követelményei

A fejlesztés elsődleges célja egy olyan kliens-szerver architektúrára épülő webalkalmazás létrehozása volt, amely áttekinthető módon támogatja az eladó és kiadó ingatlanok közzétételét és keresését. A rendszernek egyszerre kellett kezelnie a nyilvános és a hitelesített funkciókat: a publikus oldalakat bárki használhatja, bizonyos műveletekhez azonban bejelentkezésre van

szükség. Ilyen például a hirdetés feladása, a saját hirdetések kezelése, a profil frissítése vagy a kedvencek mentése.

A funkcionalitás tervezésekor fontos szempont volt, hogy a rendszer valós felhasználási helyzeteket modellezzük. Egy érdeklődő általában először szűr a hirdetések között, majd megnyitja a számára érdekes ingatlan adatlapját, megnézi a képeket, összeveti az árat és a leírást, végül kapcsolatba lép a hirdetővel. A hirdető oldaláról nézve ezzel párhuzamosan szükség van saját hirdetéslista kezelésére, képfeltöltésre, szerkesztésre és az érkező érdeklődések áttekintésére.

Követelményként jelent meg az is, hogy az adminisztrátor legalább alap szinten tudja moderálni a rendszert. Emiatt külön admin útvonal és külön admin végpontok készültek. Ezekkel lehetőség van a felhasználói szerepkörök módosítására, a hirdetések státuszának állítására és az adminnak küldött üzenetek kezelésére. Bár a projekt alapvetően oktatási célú, a jogosultságok szétválasztása és a moderációs logika megléte fontos gyakorlati elem.

A nem funkcionális követelmények között kiemelt helyen szerepelt a jól áttekinthető felület, a gyors fejlesztési ciklus, a bővíthető szerkezet és a biztonságos hitelesítés. A választott stack ezt jól támogatja: a React komponens-alapú felületet ad, az ASP.NET Core stabil API háttérrel biztosít, a Dapper közvetlen és gyors SQL alapú adatkezelést tesz lehetővé, a MySQL pedig egyszerűen kezelhető relációs adatbázisként szolgál. Emellett külön szempont volt, hogy a projekt helyi fejlesztői környezetben is könnyen indítható legyen.

3. Követelményanalízis

A rendszer funkcionális követelményei három jól elkülöníthető csoportba rendezhetők. Az első csoportba a publikus funkciók tartoznak: ide sorolható a hirdetések listázása, a részletes keresés, a találatok rendezése, a lapozás és az egyes hirdetések adatlapjának megjelenítése. Ezek a műveletek bejelentkezés nélkül is használhatók, mert a projekt egyik alapelve az volt, hogy az érdeklődő már a regisztráció előtt képet kapjon a kínálatról.

A második csoportba a hitelesített felhasználói funkciók tartoznak. A regisztráció és a bejelentkezés után elérhetővé válik a hirdetésfeladás, a saját hirdetések listázása és szerkesztése, a profiloldal, a jelszócsere, a kedvencek tárolása és a hirdetésekkel kapcsolatos kapcsolatfelvétel. Ezeknél a műveleteknél a rendszernek ellenőriznie kell a felhasználó azonosítóját és bizonyos esetekben a tulajdonjogot is, például hirdetés szerkesztése vagy törlése esetén.

A harmadik csoport az adminisztrációs funkcióké. Az admin felület célja nem egy teljes értékű vállalati dashboard létrehozása volt, hanem az alapvető felügyeleti műveletek modellezése. Ezért a rendszer támogatja a felhasználók listázását, a szerepkörök módosítását, a hirdetések moderálását és az admin üzenetek megtekintését. A jogosultságkezelés itt különösen fontos, mert nem elegendő az, hogy valaki bejelentkezett felhasználó legyen; admin művelet csak ADMIN szerepkörrel hajtható végre.

A nem funkcionális elvárások közül a reszponzív megjelenés, az egyértelmű hibakezelés, a karbantarthatóság és a bővíthetőség volt a legfontosabb. A kliensoldali felület ezért külön komponensekre és oldalakra lett bontva, a backend pedig modulonként szervezett controllereket és repository-kat használ. Ezzel a szerkezettel a rendszer könnyebben áttekinthető, és a későbbi fejlesztések - például térképes keresés, email értesítés vagy részletesebb statisztika - is egyszerűbben hozzáadhatók.

Terület	Követelmény	Megvalósítás a projektben
Hitelesítés	Csak bejelentkezett felhasználó érjen el védett műveleteket	JWT Bearer middleware és [Authorize]
Jogosultság	Admin műveletek szerepkörhöz kötöttek legyenek	Admin controller saját szerepkör-ellenőrzéssel
Adatkezelés	Szűrés és lapozás támogatása	Dapper alapú dinamikus SQL, LIMIT/OFFSET
Kliensoldal	Az oldalak SPA-n belül működjenek	React Router alapú útvonalkezelés
Karbantarthatóság	A funkciók legyenek modulok szerint szétválasztva	Auth, Listings, Profile, Admin stb.
Felhasználói élmény	A gyakori műveletek gyorsan elérhetők legyenek	Külön oldalak, kártyanézetek, jól látható navigáció

1. táblázat - Főbb követelmények és jelenlegi megvalósításuk

4. Felhasznált technológiák

A frontend felépítésében a HTML és a CSS alapvető szerepet tölt be, még akkor is, ha a kód TSX komponensekben jelenik meg. A HTML-szerű jelölés adja a felület szerkezetét: űrlapmezők, navigációs elemek, gombok, kártyák és listák formájában. A CSS felel a vizuális megjelenésért, vagyis a térközökért, a színekért, a tipográfiáért, a rugalmas elrendezésekért és a mobilbarát viselkedésért. A projekt saját stíluslapjai modern, letisztult felületet adnak, ami a mellékelt képernyőképeken is jól látszik.

A TypeScript a kliensoldali fejlesztés egyik legfontosabb eleme volt. A típuskényszer segített abban, hogy az API-ból visszaérkező adatszerkezetek kezelése megbízhatóbb legyen, és kevesebb futásidejű hiba forduljon elő. Ez különösen akkor hasznos, amikor összetettebb objektumokkal kell dolgozni, például részletes hirdetésadatokkal, paginált válaszokkal vagy admin listákkal. A frontend package.json alapján a projekt TypeScript 5.9.3 verziót használ.

A React komponensalapú felépítése lehetővé tette, hogy a visszatérő felületi elemek külön újrahazsnosítható egységekbe kerüljenek. Ilyen például a navigációs sáv, a hirdeteskártya vagy az admin útvonalvédő komponens. A route kezeléshez React Router DOM került használatra, amely a projektben a 7.13.0 verzióval szerepel. Ez a kliensoldali navigáció azért fontos, mert az alkalmazás egyoldalas webalkalmazásként működik, vagyis az oldalak közötti váltás nem teljes újratöltéssel történik.

A fejlesztői eszközök között a Vite kiemelt szerepet kapott. A projektben a fejlesztői szerver indítását, a gyors visszatöltést és a production build folyamatát is ez kezeli. Érdekesség, hogy a package.json fájlban a Vite felülbírálással, rolldown-vite csomagként szerepel, ami jól mutatja, hogy a projekt nem egy teljesen generikus sablon, hanem a fejlesztés során testreszabott környezetben készült. Emellett a frontend mappában bun.lock fájl is található, ami arra utal, hogy a projekt kipróbálta vagy használta a Bun ökoszisztémát is, még ha a napi használatot főként az npm parancsok jelentik.

A backend oldalon az ASP.NET Core Web API biztosítja a rendszer gerincét. A RealEstate.Api.csproj alapján a projekt net9.0 célkeretrendszert használ, ami korszerű .NET környezetet jelent. Az ASP.NET Core előnye, hogy jól kezeli a middleware alapú felépítést, a dependency injectiont, az autentikációs modulokat és a REST API szemléletű controller alapú végpontokat. A Program.cs fájlban jól látható a controllerek, a CORS, a Swagger, a JWT hitelesítés és az adatkapcsolatok összeállítása.

Az adat-hozzáférés Dapper segítségével valósul meg. A Dapper egy könnyű micro-ORM, amely nem rejti el a fejlesztő elől az SQL lekérdezéseket, mégis kényelmesen képes az eredményeket objektumokba leképezni. Ezt a projekt több ponton ki is használja: a hirdetések keresésénél dinamikus WHERE feltételek, rendezés és lapozás szükséges, amit a Dapperrel áttekinthetőbb megoldani, mint egy nehezebb absztrakcióra támaszkodva. A csomagverzió a projektben 2.1.66.

Az adattárolás MySQL adatbázissal történik, a szervertoldali kapcsolódást pedig a MySqlConnection 2.5.0 csomag biztosítja. A schema.sql jól mutatja, hogy a rendszer relációs modellben gondolkodik: külön táblák tárolják a felhasználókat, a hirdetéseket, a képeket, a kedvenceket, az érdeklődéseket és az admin üzeneteket. Az idegen kulcsok gondoskodnak a kapcsolatok konzisztenciájáról, például a hirdetéshez tartozó képek és érdeklődések nyilvánvalóan nem létezhetnek a hirdetés rekordja nélkül.

A hitelesítés JWT alapú. A backend a bejelentkezés után olyan tokenet állít elő, amely a felhasználó azonosítóját, email címét, nevét és szerepkörét is tartalmazza. A frontend ezt a tokenet elmenti, majd a védett API hívásoknál Authorization: Bearer fejlécben továbbítja. Ezzel a rendszer stateless módon tud hitelesíteni, vagyis nincs szükség szervertoldali session tárolásra. A Program.cs fájlban a Microsoft.AspNetCore.Authentication.JwtBearer és a System.IdentityModel.Tokens.Jwt csomagok ehhez szükséges beállításai jól követhetők.

A jelszavak védelmére BCrypt.Net-Next került használatra, a 4.0.3 verzióban. A BCrypt előnye, hogy szózott hash formátumot használ, így a jelszavak még adatbázis-szivárgás esetén sem lennének közvetlenül visszaállíthatók. A frontend és a backend közötti hálózati kommunikációhoz natív Fetch API hívások tartoznak, amelyeket a kliensoldali api modulok egységesítenek. Az API végpontok tesztelését pedig a Swagger támogatja; a projektben Swashbuckle.AspNetCore 6.5.0 szerepel, és a Program.cs külön Bearer security definíciót is létrehoz a védett végpontok kipróbálásához.

Réteg	Technológia	Verzió / szerep
Frontend	React / React DOM	19.2.0 - felhasználói felület és renderelés
Frontend	TypeScript	5.9.3 - típusbiztos klienslogika
Frontend	React Router DOM	7.13.0 - útvonalkezelés
Frontend	Vite	rolldown-vite 7.2.5 - dev szervert és build
Frontend	Fetch API	HTTP kommunikáció az API-val
Backend	.NET / ASP.NET Core Web API	net9.0 - REST végpontok és middleware
Backend	Dapper	2.1.66 - SQL alapú adat-hozzáférés
Backend	MySqlConnection	2.5.0 - MySQL kapcsolat
Backend	JWT Bearer	tokenes hitelesítés és jogosultságkezelés
Backend	BCrypt.Net-Next	4.0.3 - jelszóhash-elés
Backend	Swashbuckle / Swagger	6.5.0 - API tesztelés
Adatbázis	MySQL / MariaDB	relációs adattárolás

2. táblázat - A projekt technológiai stackje

5. Projektstruktúra és rendszerfelépítés

A projekt felépítésének egyik legnagyobb előnye az, hogy a frontend, a backend és az adatbázishoz kapcsolódó fájlok már a legfelső szinten elkülönülnek egymástól. A gyökérkönyvtárban megtalálható a backend, a frontend és a db mappa, valamint a docker-compose.yml fájl is, ami azt mutatja, hogy a rendszer később akár konténeres formában is futtatható. Ez a tagolás már önmagában sokat segít a karbantarthatóságon, mert rögtön látszik, hogy melyik részért melyik mappa felel.

A backend/RealEstate.Api mappa a szerveroldali alkalmazás központja. Itt külön modulokba szervezve szerepel az Auth, a Listings, az Inquiries, a Profile, az Admin és az AdminMessages funkcionalitás. Az adat-hozzáférést a Data mappa repository osztályai kezelik, a képfeltöltéshez az Images alatti LocalImageStorage tartozik, az általános hibakezelést pedig a Common mappa biztosítja. Ez a moduláris szervezés sokkal áttekinthetőbbé teszi a projektet, mintha minden controller és DTO egyetlen nagy mappába került volna.

A frontend/src mappában hasonlóan rendezett felépítés látható. Az api könyvtárban a backendhez tartozó kliensoldali hívások szerepelnek, az auth alatt a session kezelés, a components mappában a közös felületi elemek, a pages alatt pedig az önálló route szintű oldalak. A külön FavoritesPage, CreateListingPage, ListingDetailsPage, MyListingsPage vagy AdminPage megléte jól mutatja, hogy a rendszer nem egyetlen hosszú komponenst használ, hanem tudatosan kisebb, jobban karbantartható egységekre bontja a felületet.

Rendszerszinten a projekt három rétegre bontható. A megjelenítési réteget a böngészőben futó React kliens adja, az üzleti logikát és az API végpontokat az ASP.NET Core backend biztosítja, az adattárolás pedig MySQL adatbázisban történik. A kliens HTTP kéréseket küld az API-nak, az API ellenőrzi a jogosultságokat, végrehajtja a validációt és repository-kon keresztül olvas vagy ír az adatbázisba. Ez a háromrétegű felépítés a projekt méretéhez jól illeszkedik, és bővíthetőség szempontjából is jó alap.

Projektstruktúra

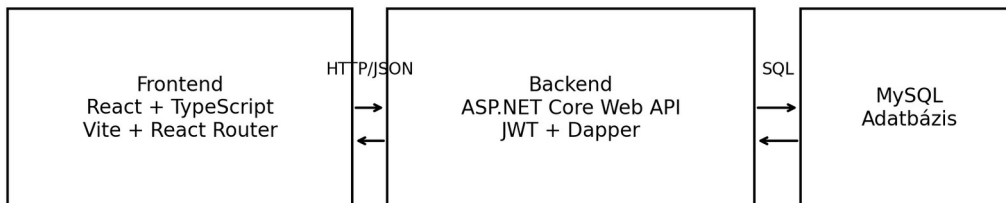
```

realestate-app/
├─ backend/
│  └─ RealEstate.Api/
│     ├── Auth/
│     ├── Listings/
│     ├── Inquiries/
│     ├── Profile/
│     ├── Admin/
│     ├── AdminMessages/
│     ├── Data/
│     ├── Images/
│     ├── Program.cs
│     └─ appsettings.json
├─ frontend/
│  ├── src/
│  │  ├── api/
│  │  ├── auth/
│  │  ├── components/
│  │  ├── pages/
│  │  └─ utils/
│  ├── package.json
│  └─ vite.config.ts

```

1. ábra - A projekt fő mappaszerkezete

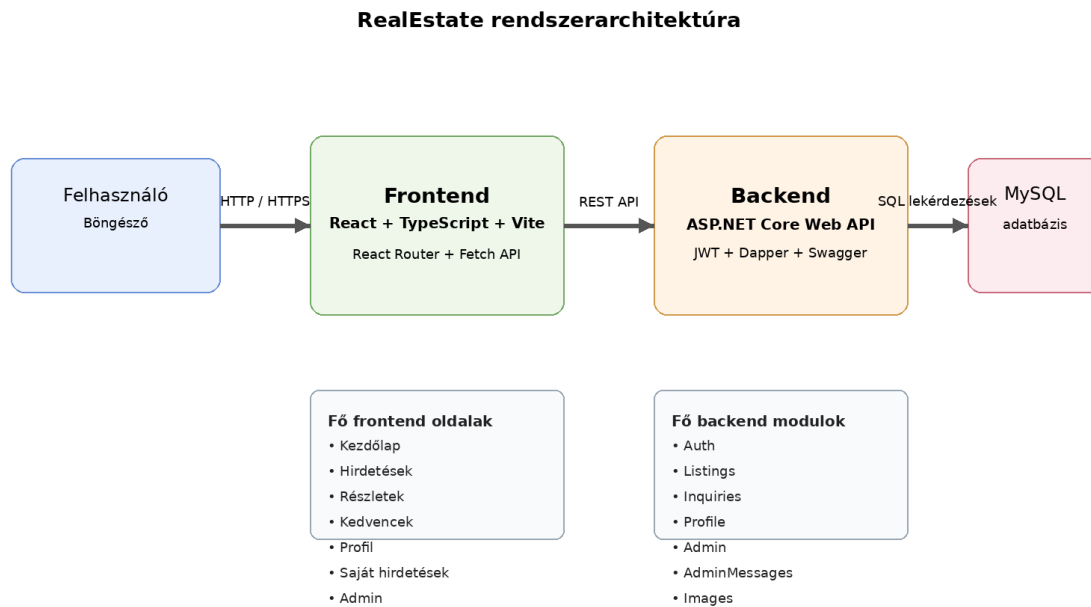
A projektstruktúra ábrája jól mutatja, hogy a backend és a frontend nemcsak logikailag, hanem mappaszerkezet szintjén is elkülönül. Ez különösen akkor hasznos ha több fejlesztő dolgozna a kódbázison, vagy ha a rendszert éles környezetre kellene előkészíteni.



A rendszer 3 rétegre bontva: kliens (UI), szerver (API), adatbázis.

2. ábra - Egyszerűsített háromrétegű rendszerkép

Az egyszerűsített rendszerkép a projekt magját foglalja össze: a kliens oldalon React + TypeScript + Vite fut, a szerver oldalon ASP.NET Core Web API és Dapper dolgozik, az adattárolás pedig MySQL-ben történik.



3. ábra - Részletes rendszerarchitektúra és fő modulok

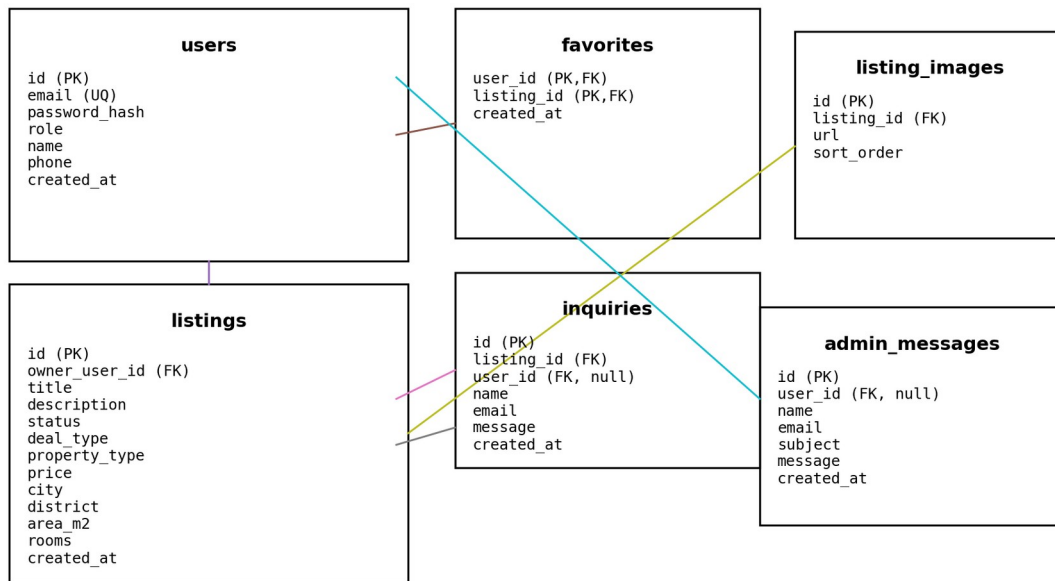
6. Adatbázisterv és adatszerkezet

A rendszer adatbázisa hat fő táblára épül: users, listings, listing_images, favorites, inquiries és admin_messages. Ez a felosztás közvetlenül a funkcionális követelményekből következik. A felhasználók külön táblában szerepelnek, a hirdetések szintén önálló entitást alkotnak, a képek és a kedvencek kapcsolati logika miatt külön táblába kerültek, az érdeklődések és az admin felé küldött üzenetek pedig külön kommunikációs adattáblaként jelennek meg.

A users tábla a felhasználó alapadatait és hitelesítési adatait tárolja. Ide tartozik az email cím, a password_hash, a role, a név, a telefonszám és a létrehozás időpontja. A schema.sql szerint az email mező egyedi kulcsot kapott, így ugyanazzal az email címmel nem lehet két külön fiókot létrehozni. A role mezőben USER, AGENT és ADMIN értékek jelenhetnek meg, még akkor is, ha a jelenlegi felület főként a USER és ADMIN szerepkört használja.

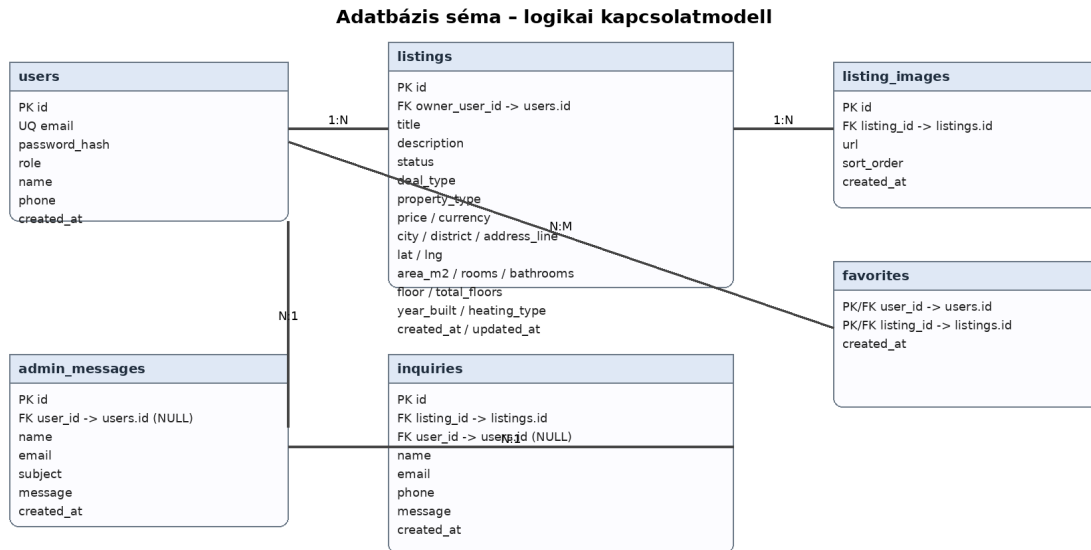
A listings tábla az alkalmazás központi üzleti entitása. Egy hirdetéshez tartozik tulajdonos, cím, leírás, státusz, ügylet típusa, ingatlan típusa, ár, pénznem, város, kerület, cím, alapterület, szobaszám és több egyéb paraméter. Külön figyelmet érdemel, hogy a séma lat és lng mezőket is tartalmaz. Ez a jelenlegi verzióban még nincs teljesen kihasználva, de erős alapot jelent egy későbbi térképes kereséshez.

A listing_images tábla külön kezeli a képeket. Ez adatbázis-tervezési szempontból jó döntés, mert egy hirdetéshez több kép is tartozhat, és ezek sorrendjét is meg kell őrizni. A favorites klasszikus kapcsolótábla a users és a listings között, összetett elsődleges kulccsal. Az inquiries a hirdetésekhez kapcsolódó érdeklődéseket tárolja, míg az admin_messages a főoldalról küldhető általános admin üzeneteket. A külső kulcsok és az ON DELETE szabályok gondoskodnak arról, hogy az adatok ne maradjanak árva állapotban.



Egyszerűsített ER diagram (a projekt schema.sql alapján).

4. ábra - Egyszerűsített ER diagram a projekt fő tábláival



5. ábra - Részletes logikai adatbázisdiagram

Tábla	Fő mezők	Szerep
users	id, email, password_hash, role, name, phone	felhasználói és hitelesítési adatok
listings	title, description, status, deal_type, property_type, price, city	ingatlanhirdetések törzsadatai
listing_images	listing_id, url, sort_order	hirdetéshez kapcsolódó képek
favorites	user_id, listing_id	kedvencek kapcsolótábla
inquiries	listing_id, user_id, name, email, message	hirdetésekre érkező érdeklődések
admin_messages	user_id, name, email, subject, message	adminnak küldött üzenetek

3. táblázat - Az adatbázis fő táblái és szerepük

7. Backend megvalósítás

A backend indulását a Program.cs állítja össze. Itt történik a controllerek felvétele, a Swagger engedélyezése, a frontendhez igazított CORS szabály létrehozása, a MySQL kapcsolat konfigurálása és a szükséges repository-k, illetve szolgáltatások regisztrálása. A projekt külön scoped életciklussal veszi fel a MySqlConnection példányt, amelyet a repository osztályok használnak. Ugyanitt kerül beállításra a JWT hitelesítés és az Authorization middleware is.

Az Auth modul a regisztrációt, a bejelentkezést és a felhasználóhoz tartozó tokengenerálást kezeli. A regisztráció ellenőrzi, hogy az email cím foglalt-e, majd a jelszót BCrypt hash formára alakítja és elmenti. Bejelentkezéskor a rendszer a tárolt hash és a megadott jelszó összevetésével dönt a belépésről, siker esetén pedig JWT tokenet ad vissza. A token a felhasználó azonosítója mellett a szerepkört és a nevet is tartalmazza, ami a kliensoldali felületnek is hasznos.

A Listings modul a projekt legfontosabb backend része. A publikus GET /api/listings végpont szűréssel, rendezéssel és lapozással dolgozik, ezért a ListingRepo dinamikus SQL építést használ. Emellett külön végpont tartozik az egyedi hirdetés lekéréséhez, a saját hirdetések listázásához, új hirdetés létrehozásához, módosításhoz, törléshez és képfeltöltéshez. Külön előny, hogy a módosító műveletek a tulajdonosi ellenőrzést is figyelembe veszik, vagyis más felhasználó nem tud idegen hirdetést módosítani.

Az Inquiries modul hirdetésenként kezeli az érdeklődő üzeneteket. A POST /api/listings/{listingId}/inquiries végpont vendégként és bejelentkezett állapotban is használható, így a user_id mező csak akkor kerül kitöltésre, ha a felhasználó hitelesítve van. A Profile modul a saját adatok kezelését valósítja meg: támogatja a név és telefonszám módosítását, valamint a jelszócserét. A projekt egyik praktikus megoldása, hogy profilfrissítés után a backend új JWT tokenet küld vissza, így a kliensoldali név és szerepkör azonnal frissülhet.

Az Admin és az AdminMessages modulok a felügyeleti műveleteket fedik le. Az admin controller nemcsak [Authorize] attribútummal védi a végpontokat, hanem a szerepkört külön is ellenőrzi. Az admin képes felhasználólistát lekérni, szerepkört módosítani, hirdetéslistát szűrni és a státuszt ACTIVE vagy INACTIVE értékre váltani. Az admin üzenetek külön végpontot kapnak: küldésük publikus, de listázásuk és törlésük kizárólag admin joggal történik.

Modul	Fő végpontok	Feladat
Auth	/api/auth/register, /login, /me	regisztráció, login, token és user állapot
Listings	/api/listings, /mine, /{id}, /{id}/images	hirdetések listázása, CRUD és képek
Inquiries	/api/listings/{id}/inquiries, /api/inquiries/mine	érdeklődések mentése és lekérése
Profile	/api/profile, /api/profile/password	profiladatok és jelszó kezelése
Admin	/api/admin/users, /api/admin/listings	szerepkörök és listing státuszok
AdminMessages	/api/admin-messages, /api/admin/messages	admin üzenetek küldése és kezelése

4. táblázat - A backend főbb moduljai és végpontjai

8. Frontend megvalósítás

A kliensoldali alkalmazás központja az App.tsx, ahol a BrowserRouter és a Routes segítségével a teljes útvonalstruktúra felépül. Itt külön route tartozik a főoldalhoz, a nyilvános hirdetéslistához, a részletes adatlaphoz, a regisztrációhoz és bejelentkezéshez, valamint a védett oldalakhoz, például a profilhoz, a kedvencekhez, a saját hirdetésekhez és az admin felülethez. Ez a felépítés jól átlátható és könnyen bővíthető.

A frontend API kommunikációja külön modulokba szerveződik. Az api/client.ts kezeli az alap URL-t, a JSON fejléceket, a token hozzáadását és a hibaüzenetek egységes feldolgozását. Erre épülnek rá a külön auth.ts, listings.ts, inquiries.ts, profile.ts, admin.ts és adminMessages.ts modulok. Ezzel a megoldással az oldalkomponensekben kevesebb ismétlődő hálózati logika jelenik meg, a kód rövidebb és karbantarthatóbb marad.

A felhasználói élmény szempontjából a Navbar kulcsfontosságú komponens. A képernyőképek alapján is látható, hogy a menüsor a gyakran használt oldalak között gyors navigációt biztosít, és bejelentkezés után megjeleníti az admin státuszt is. A felület több ponton kártyás megjelenítést használ: a főoldali statisztikáknál, a hirdetéslistában és a gyors kereső blokknál is. Ez az irány vizuálisan rendezettebbé teszi az oldalt, mint egy egyszerű, táblázatszerű listázás.

A hirdetések megjelenítésében fontos szerepet kap a kliensoldali állapotkezelés. A szűrőmezők, a találati oldal, a rendezési mód és az oldalméret mind befolyásolják a lekérdezést, a felületnek pedig ezeket jól követhető módon kell visszajeleznie. A projektben a részletes kereső több mezőt kombinál, mégis átlátható marad, ami kifejezetten fontos egy ingatlanos rendszernél, ahol sok adat alapján kell szűrni.

A kedvencek és a session kezelés localStorage alapú megközelítést használ. Ez egyszerű, de a projekt céljához jól illeszkedő megoldás. A bejelentkezett felhasználó állapota így újratöltés után is megmaradhat, a felület pedig rögtön tudja, hogy mely menüpontokat kell megjeleníteni. Ugyanez igaz az olvasott érdeklődések kliensoldali jelölésére is: a rendszer nem vállal fölösleges szerveroldali bonyolítást, mégis használható élményt ad.

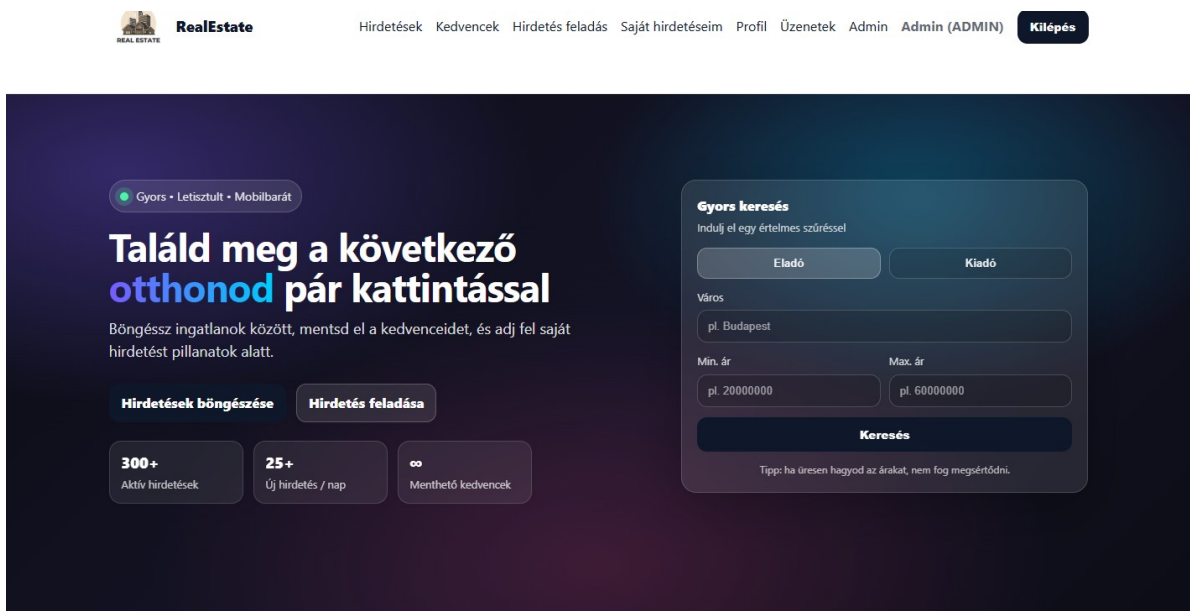
Útvonal	Oldal	Rövid leírás
/	HomePage	kezdőlap, gyorskereső, admin üzenetküldés
/listings	ListingsPage	szűrhető, rendezhető listanézet
/listings/:id	ListingDetailsPage	adatlapp, képek, érdeklődés

/create	CreateListingPage	új hirdetés feladása
/my-listings	MyListingsPage	saját hirdetések listája
/favorites	FavoritesPage	kedvencek gyűjtőoldala
/profile	ProfilePage	profil és jelszócsere
/inquiries	MyInquiriesPage	érdeklődések megjelenítése
/admin	AdminPage	admin funkciók

5. táblázat - A frontend legfontosabb útvonalai

9. Felhasználói dokumentáció

A rendszer használata a főoldaltól indul. Itt egy jól látható gyors kereső segíti a tájékozódást, a bal oldali nagy szöveges blokk pedig röviden megfogalmazza az alkalmazás célját. A főoldal kialakítása vizuálisan modernebb, sötétebb tónusú felületet használ, ami már első ránézésre elkülöníti a projektet egy teljesen alap sablontól. A statisztikai kártyák ugyan nem kötődnek közvetlenül adatbázis-lekérdezéshez, mégis jó bevezető elemek, mert irányt adnak a felhasználónak.

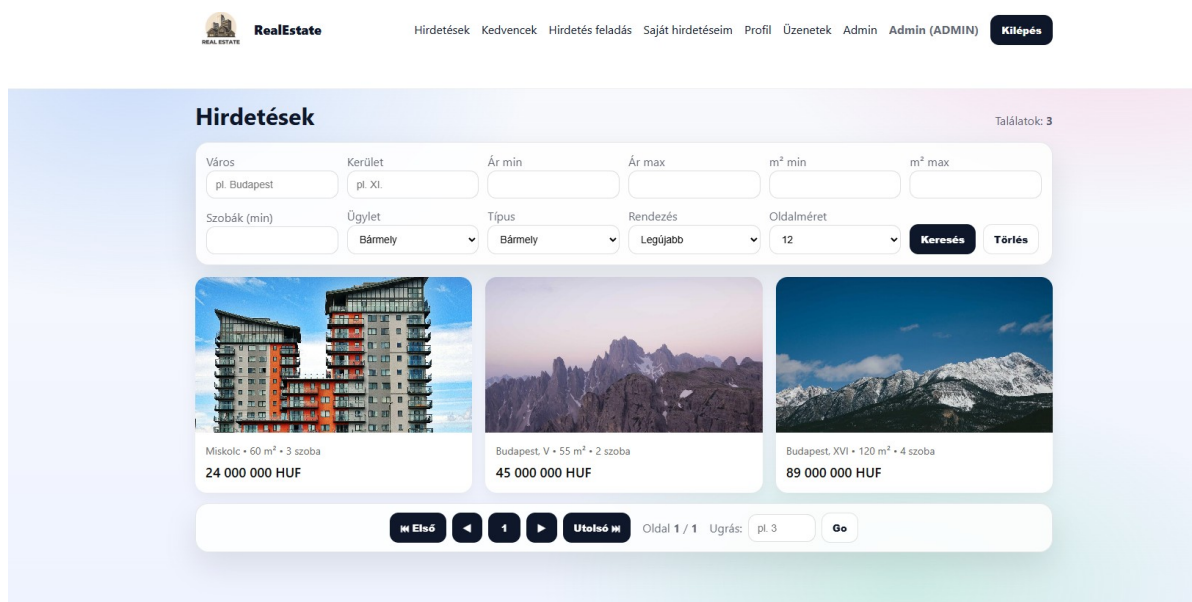


6. ábra - A főoldal gyors keresővel és kiemelt funkciókkal

A regisztráció és a bejelentkezés a rendszer alapfolyamatai közé tartozik. Regisztrációkor a felhasználó megadja a nevét, az email címét és a jelszavát, majd a backend létrehozza a fiókot. Sikeres művelet után a rendszer rögtön token-t is adhat vissza, így a felhasználó külön

bejelentkezés nélkül folytathatja a használatot. Bejelentkezéskor email és jelszó szükséges; hibás adatok esetén a rendszer jól érthető hibaüzenetet küld vissza.

A Hirdetések oldalon a részletes szűrés a rendszer egyik legerősebb pontja. A képernyőkép alapján egyszerre kezelhető a város, a kerület, az árminimum és ármaximum, az alapterület, a szobaszám, az ügylet típusa, az ingatlan típusa és a rendezés. A lapozás külön navigációs blokkban jelenik meg, ami akkor hasznos, ha több oldalnyi találat keletkezik. A kártyás megjelenítés gyors áttekintést ad a címről, az árról, a városról és az alap paramétereiről.



7. ábra - Hirdetési lista részletes szűréssel és lapozással

A részletes hirdetésadatlap logikailag a listanézet folytatása. Itt a felhasználó nagyobb képeken láthatja az ingatlant, elolvashatja a teljes leírást, megnézheti az alapadatokat, majd eldöntheti, hogy menti-e kedvencnek vagy kapcsolatba lép a hirdetővel. A kapcsolatfelvétel űrlapon keresztül történik, amely az inquiries táblába mentett üzenetet hoz létre. Bejelentkezett felhasználó esetén a háttérben a user_id is tárolódik, ami később segíti a saját üzenetek és érdeklődések visszakeresését.

A Hirdetés feladása oldal csak belépés után érhető el. Az űrlap a cím, a város, a kerület, az ügylet típus, az ingatlan típus, az ár, az alapterület, a szobaszám és a leírás megadását teszi lehetővé, majd külön blokkban kezeli a képek feltöltését. A mellékelt képernyőkép alapján a felület legfeljebb nyolc képet enged kezelni, és drag-and-drop jellegű élményt kínál. Ez funkcionálisan és megjelenésében is jól illeszkedik a projekt témájához.

Hirdetés feladása

Hirdetés címe

Város

Kerület (opcionális)

Eladó

Lakás

Ár (HUF)

Alapterület (m²)

Szobák száma (pl. 2.5)

Leírás

Képek

Húzd rá a képeket, vagy válaszd ki. Utána fogd-és-vidd a sorrendhez. (max 8)

Kiválasztás

Dobd ide a képeket

PNG / JPG / WEBP

0/8

Hirdetés mentése

8. ábra - Hirdetés feladása űrlap és képfeltöltési felület

A Saját hirdetésem oldal arra szolgál, hogy a felhasználó egy helyen lássa az összes általa feladott ingatlant. Innen indul a szerkesztés és a törlés is. A backend itt tulajdonjogi ellenőrzést végez, ezért a művelet csak a hirdetés tulajdonosának sikerülhet. A Kedvencek oldal hasonlóan fontos kényelmi funkció: a felhasználó egy kattintással elmentheti a számára érdekes hirdetéseket, majd később visszatérhet hozzájuk. A Profil oldalon a személyes adatok és a jelszó kezelése történik, míg az Üzenetek oldalon a hirdetésekhez kapcsolódó érdeklődések jelennek meg.

10. Adminisztrátori funkciók

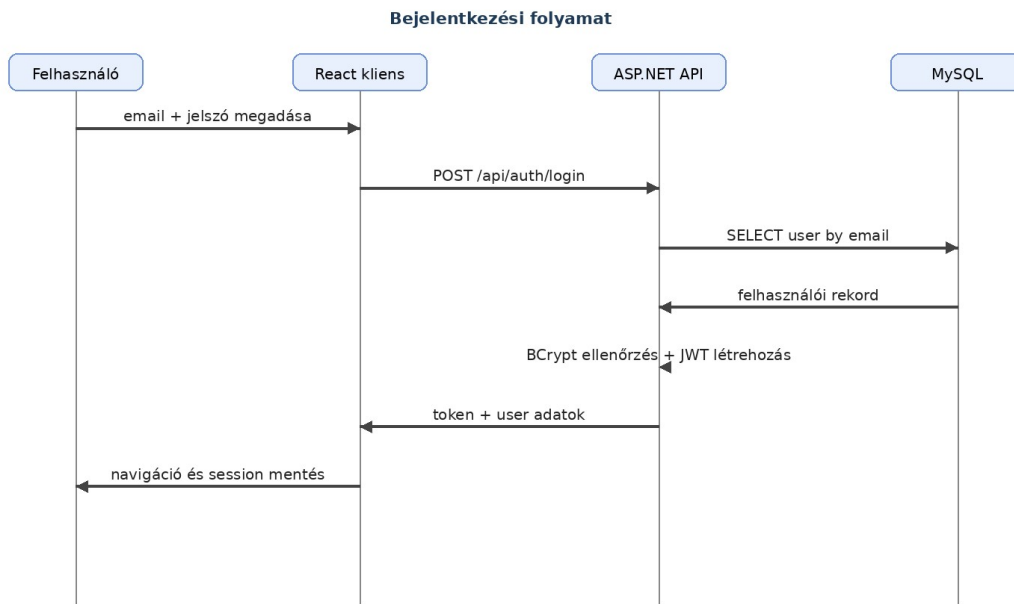
Az admin felület célja, hogy a rendszer működtetéséhez szükséges minimális, de lényeges felügyeleti funkciókat egy helyre gyűjtse. Az admin oldalon külön szekcióban jelennek meg a felhasználók, a hirdetések és az admin üzenetek. Ez a felosztás logikus, mert egymástól eltérő adminisztrációs feladatokról van szó: az egyik a fiókokhoz, a másik a tartalomhoz, a harmadik pedig az üzemeltetői kommunikációhoz kapcsolódik.

A felhasználók listázásánál az admin megtekintheti az alapadatokat és szerepkört módosíthat. Ez fejlesztési és tesztelési szempontból különösen hasznos, mert gyorsan modellezhető vele a USER és ADMIN jogosultság közötti különbség. A hirdetéseknel az admin a státuszt ACTIVE és

INACTIVE értékek között állítja, ami egy egyszerű, mégis jól használható moderációs logika. Az admin üzeneteknél az oldalon a látogatóktól vagy felhasználóktól érkező megkeresések listázhatók és törölhetők.

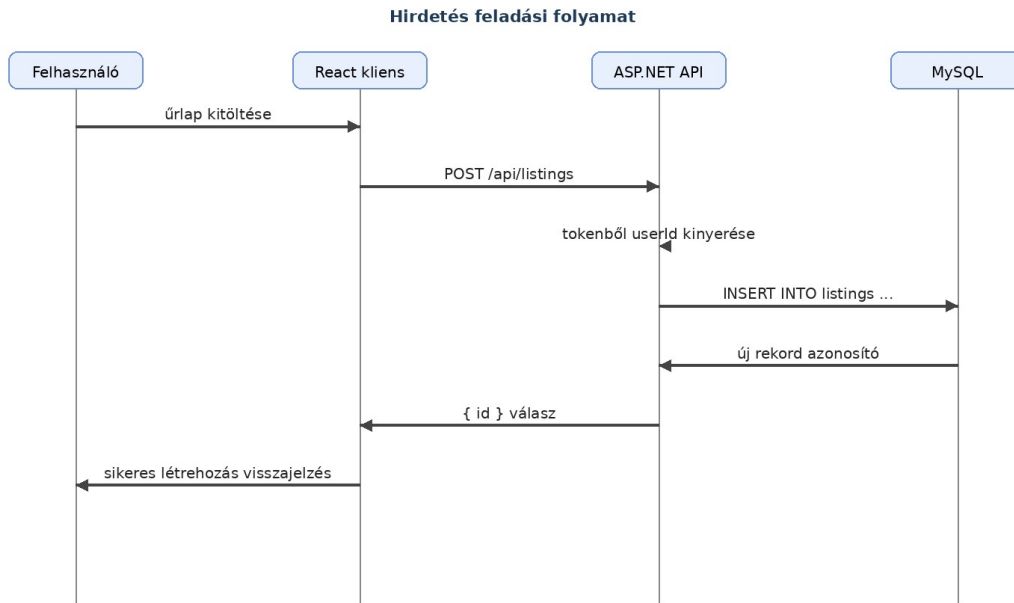
11. Működési forgatókönyvek

A bejelentkezési folyamat során a felhasználó a kliensoldali űrlapon megadja az email címét és a jelszavát. A frontend POST kérést küld az /api/auth/login végpontra, a backend pedig email alapján lekéri a felhasználót, ellenőrzi a BCrypt hash-t, majd siker esetén JWT tokenet hoz létre. A token és a felhasználói adatok visszakérülnek a klienshez, amely eltárolja ezeket és frissíti a navigációs állapotot. A folyamat egyszerű, de jól mutatja a frontend, a backend és az adatbázis együttműködését.



9. ábra - Bejelentkezési folyamat a kliens, az API és az adatbázis között

A hirdetés létrehozásának menete hasonlóan jól szemlélteti a rendszer logikáját. A felhasználó kitölti az űrlapot, a kliens elküldi az adatokat a listings végpontra, a backend a tokenből kiolvassa a felhasználó azonosítóját, majd új rekordot hoz létre a listings táblában. Ezután a képfeltöltés külön végponttal történik, így a hirdetés és a képek kezelése szétválasztható. Ez a döntés adatkezelési és hibakeresési szempontból is előnyös.



10. ábra - Hirdetés létrehozásának egyszerűsített folyamata

Az érdeklődéskezelési folyamatban a felhasználó vagy látogató a részletes adatlapról küld üzenetet. A backend elmenti a bejegyzést az inquiries táblába, majd a hirdetés tulajdonosa később a saját üzenetek között visszaneézheti azt. Az admin moderációs folyamat ennél kicsit más: ott a kliensoldali kezelőfelület lekéri a listákat, az admin módosítást indít, a backend ellenőrzi a szerepkört és végrehajtja az SQL frissítést.

12. Biztonsági megoldások és hibakezelés

A projekt biztonsági megoldásainak középpontjában a jelszóvédelem, a tokenes hitelesítés és a jogosultság-ellenőrzés áll. A jelszavak BCrypt hash formában kerülnek eltárolásra, vagyis a rendszer soha nem ment nyers jelszót az adatbázisba. Ez alapvető követelmény minden olyan alkalmazásnál, ahol felhasználói fiókokkal kell dolgozni. Bejelentkezéskor a backend a hash és a megadott jelszó összehasonlításával dönt a hitelesítésről.

A JWT token alapú működés lehetővé teszi, hogy a szerver állapotmentes módon azonosítsa a felhasználót. A tokenből kiolvasható a sub, az email, a role és a name claim, ezért az API sok esetben további lekérdezés nélkül is tud dönteni arról, hogy egy művelet végrehajtható-e. Ugyanakkor a valóban érzékeny műveleteknél - például egy hirdetés szerkesztésénél vagy törlésénél - a rendszer az adatbázis szintjén is ellenőriz, így a kliensoldali manipuláció nem lenne elég a jogosulatlan módosításhoz.

A hibakezelésben fontos szerepet kap az ApiExceptionFilter. Ennek köszönhetően a backend nem széteső, összeviszza hibaüzeneteket küld vissza, hanem egységesen kezeli a tipikus üzleti hibákat. Ilyen lehet a foglalt email cím, a rossz jelszó, a hiányzó feltöltött fájl vagy a jogosultsági probléma. Fejlesztési szempontból különösen hasznos a /dbcheck végpont is, amely gyorsan megmutatja, hogy az API valóban eléri-e az adatbázist.

13. Fejlesztői környezet, indítás és tesztelés

A projekt helyi futtatásához .NET 9 SDK, Node.js vagy kompatibilis JavaScript futtatókörnyezet, valamint MySQL vagy MariaDB adatbázis szükséges. A backend a RealEstate.Api projektből indítható, a frontend a frontend mappából futtatható Vite fejlesztői szerverrel. Az adatbázis létrehozásához a db/schema.sql, illetve igény szerint a reset.sql és a seed.sql használható. A gyökérkönyvtárban elhelyezett docker-compose.yml arra utal, hogy a projektet később összefogottabban, konténeresen is lehet majd futtatni.

A napi fejlesztői munka szempontjából a legfontosabb lépések a függőségek telepítése, az adatbázis előkészítése, a backend indítása és a frontend indítása. A backend a launchSettings alapján HTTP és HTTPS címen is elérhető, a frontend pedig a Vite alapértelmezett portján fut. A Program.cs CORS szabálya kifejezetten számol a localhost:5173 eredettel, ami azt mutatja, hogy a projektben a frontend és a backend külön folyamatként futott fejlesztés közben.

A teszteléshez kényelmes eszközt ad a Swagger UI, mert az egyes végpontok közvetlenül kipróbálhatók, és a JWT Bearer token is megadható. Emellett fontos az integrációs tesztelés is, vagyis annak ellenőrzése, hogy a felület megfelelően reagál-e a backend válaszára. A projekt esetében különösen fontos tesztesetek a regisztráció, a bejelentkezés, a publikus keresés, az új hirdetés létrehozása, a képfeltöltés, a profilfrissítés és az admin jogosultsági ellenőrzések.

Teszt ID	Teszt eset	Elvárt eredmény
T1	Regisztráció új email címmel	Sikeres válasz és session létrejött
T2	Bejelentkezés hibás jelszóval	401 hiba és érthető hibaüzenet
T3	Publikus listings szűrés	Csak a feltételnek megfelelő aktív elemek jelennek meg
T4	Új hirdetés mentése	Új rekord jön létre a listings táblában
T5	Érdeklődés küldése vendégként	Rekord jön létre user_id nélkül
T6	Profil mentése	Név vagy telefon frissül és új token érkezik
T7	Admin szerepkör csere	A users.role mező módosul
T8	Admin státuszváltás	A hirdetés státusza frissül

6. táblázat - Mintatesztek a rendszer ellenőrzéséhez

14. Továbbfejlesztési lehetőségek

A jelenlegi állapot már használható, de több irányban is továbbfejleszthető. A legkézenfekvőbb bővítés a térképes keresés lenne, hiszen a listings táblában a földrajzi koordináták mezői már most rendelkezésre állnak. Egy térképes komponenssel a felhasználó nemcsak listából, hanem vizuális elhelyezkedés alapján is választhatna.

Érdemes lenne továbbfejleszteni a kommunikációs és értesítési funkciókat is. Jelenleg az admin üzenetek és a hirdetésekhez kapcsolódó érdeklődések alap szinten működnek, de később olvasott státusz, válaszadás, email értesítés vagy akár valós idejű jelzés is beépíthető lenne. Ugyanez igaz a kedvencek és a mentett keresések fejlettebb kezelésére is.

Üzemeltetési oldalról szintén több lehetőség látszik. A konténeres futtatás, az automatizált build, a részletes naplózás, a CI/CD pipeline, a migrációkezelés és a képek optimalizált tárolása mind olyan irányok, amelyek a projektet egy komolyabb, éles környezethez közelebb vinnék. A jelenlegi moduláris szerkezet kifejezetten kedvez ezeknek a bővítéseknek.

15. Összegzés

A RealEstate projekt egy olyan teljes értékű webalkalmazás, amely jól szemlélteti a modern full-stack fejlesztés főbb elemeit. A frontend és a backend világosan elkülönül, mégis jól együttműködik, az adatbázis séma követi az üzleti logikát, a felhasználói felület pedig nemcsak funkcionálisan, hanem megjelenésében is rendezett képet ad.

A rendszer bemutatása egyszerre technikai és felhasználói nézőpontból is követhető: látható a technológiai háttér, a projektstruktúra, az adatmodell, a főbb folyamatok és a kezelőfelületek működése. Véleményem szerint ez a projekt jó alapot ad további fejlesztésekhez, és mert a látvány mögött valódi működés és több rétegben végiggondolt megvalósítás áll.

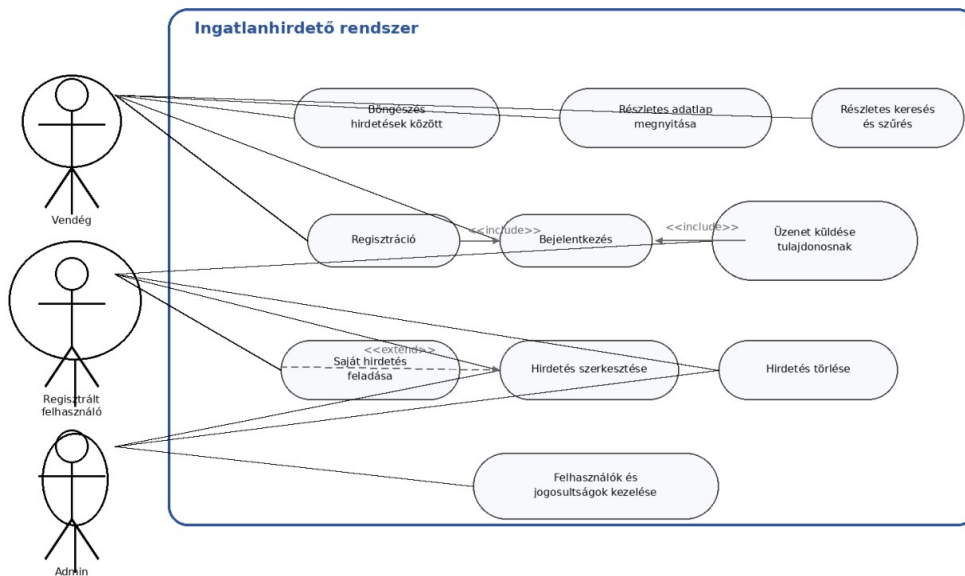
16. UML diagramok

Ebben a mellékletben a rendszerhez készített UML diagramok szerepelnek, melyek a felhasználási eseteket, az osztálykapcsolatokat, a hirdetés állapotváltozásait, az aktivitási folyamatot és a komponensek együttműködését mutatják be.

- 1. Use-case diagram
- 2. Osztálydiagram
- 3. Állapotdiagram
- 4. Aktivitásdiagram
- 5. Komponensdiagram

Use-case diagram

Szereplők: vendég, regisztrált felhasználó és admin. A diagram a rendszer legfontosabb használati eseteit és kapcsolatait mutatja.

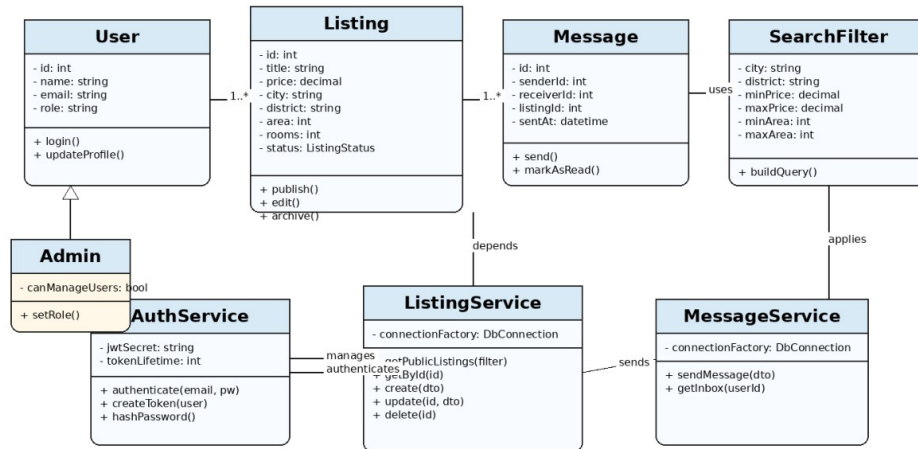


11. ábra - Use-case diagram

Osztálydiagram

A domain objektumok és a főbb szolgáltatásosztályok kapcsolatait ábrázolja.

Osztálydiagram - főbb domain és szolgáltatás osztályok

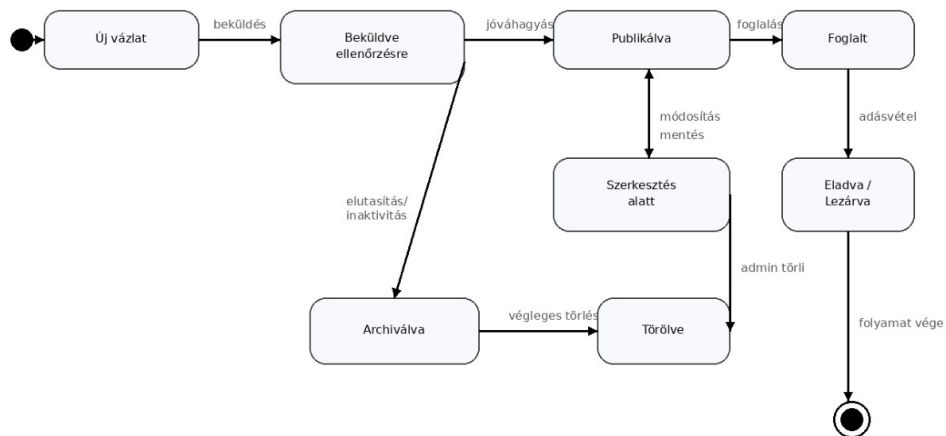


12. ábra – Osztálydiagram

Állapotdiagram

A hirdetés életciklusát mutatja a létrehozástól a lezárásig.

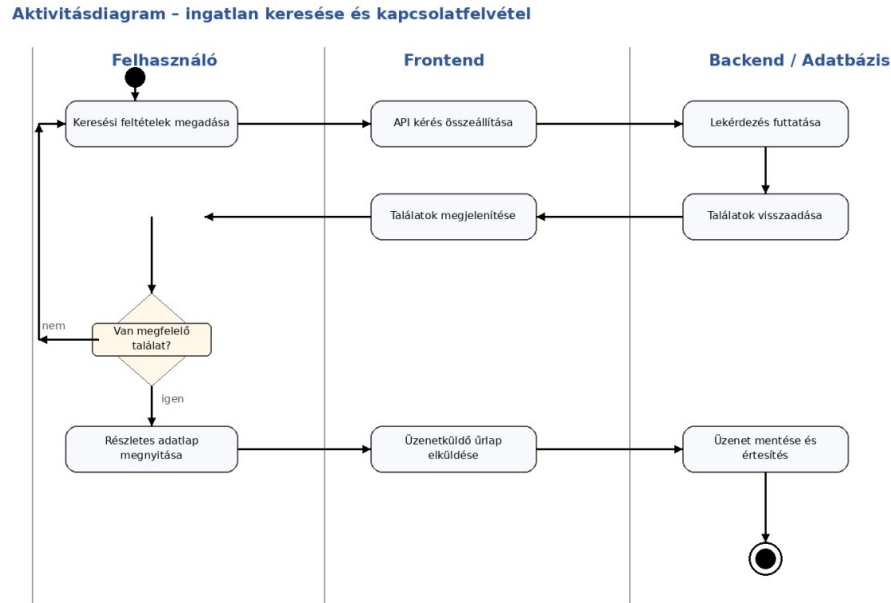
Állapotdiagram - hirdetés életciklusa



13. ábra - Állapotdiagram

Aktivitásdiagram

Az ingatlan keresése és az üzenetküldés fő folyamata látható.

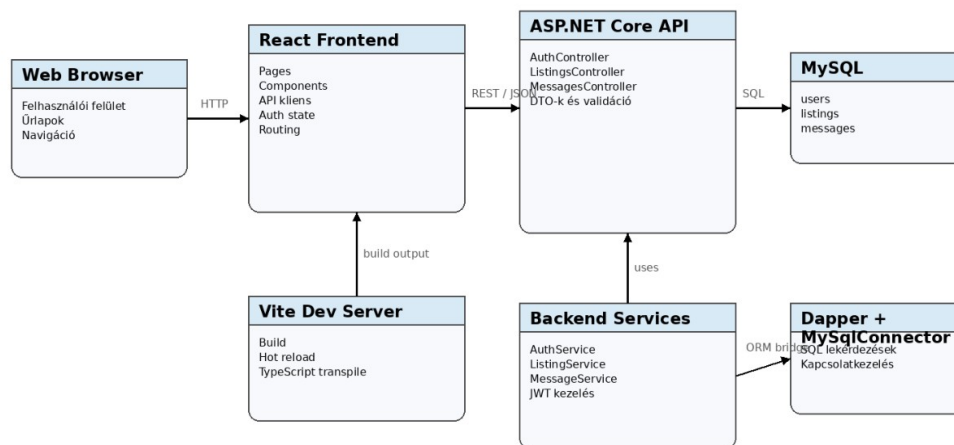


14. ábra – Aktivitásdiagram

Komponensdiagram

A frontend, backend és adatbázis komponensek együttműködését szemlélteti.

Komponensdiagram - a rendszer fő szoftverkomponensei



15. ábra - Komponensdiagram

Forrásjegyzék

1. Andrew Lock: ASP.NET Core in Action, Third Edition. Manning Publications, 2024.
2. Alex Banks - Eve Porcello: Learning React, 2nd edition. O'Reilly Media, 2020.
3. Boris Cherny: Programming TypeScript. O'Reilly Media, 2019.
4. Alan Beaulieu: Learning SQL, 3rd edition. O'Reilly Media, 2020.
5. Adam Freeman: Pro ASP.NET Core 7, 10th edition. Apress, 2023.
6. Antonio Mele: Full Stack React Projects, 2nd edition. Packt Publishing, 2020.
7. MySQL Reference Manual. Oracle.
8. Microsoft Learn - ASP.NET Core dokumentáció.
9. Microsoft Learn - JWT Bearer Authentication dokumentáció.
10. React hivatalos dokumentáció.
11. TypeScript hivatalos dokumentáció.
12. MDN Web Docs - Fetch API dokumentáció.
13. OpenAPI Specification / Swagger dokumentáció.
14. IETF RFC 7519 - JSON Web Token (JWT).
15. A mellékelt RealEstate projekt forráskódja: package.json, RealEstate.Api.csproj, Program.cs, controller és repository fájlok, db/schema.sql.